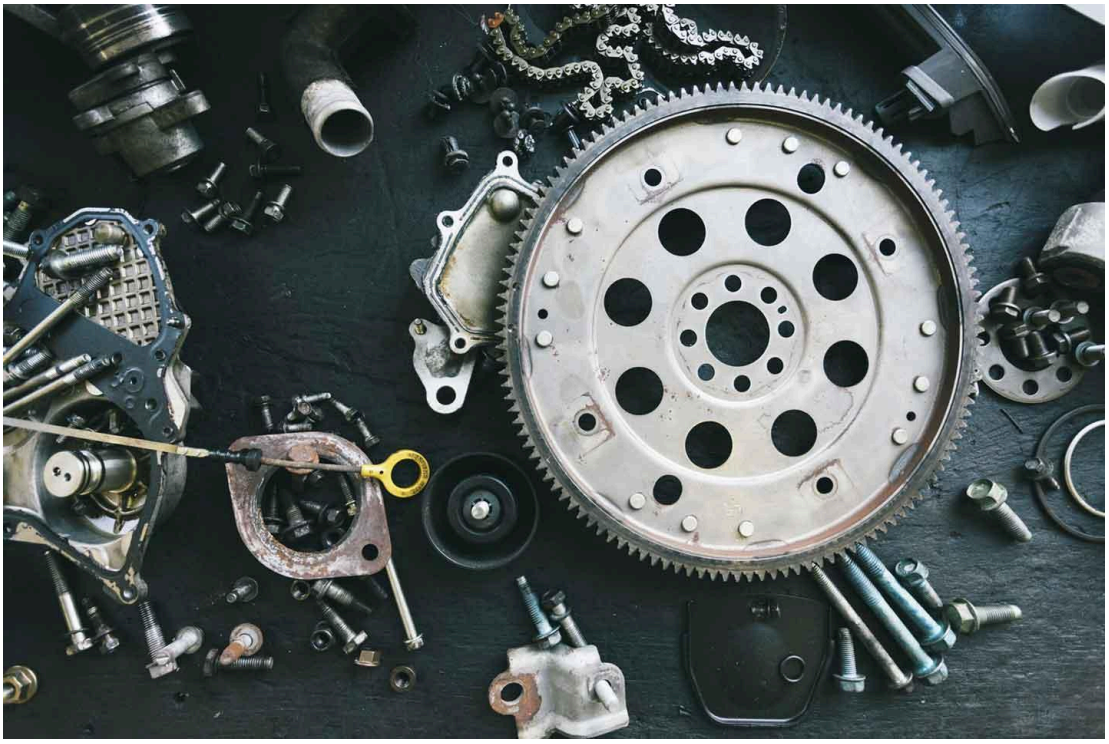
Tema 5

Relaciones Uno a Uno

Digitech
FP OFICIAL DIGITAL & TECH



Francisco Belda

ÍNDICE

Teoría. Relaciones Uno a Uno en JPA **3**
Conceptos clave en JPA: 3
Simplificando con Lombok 3
Integración de Lombok **4**
Paso a Paso: Implementación de la Relación 1a1 **5**
Creación de la entidad PerfilAcademico con Lombok 5
Modificación de la entidad Estudiante con Lombok 6
Explicación técnica 8
Actualización del Controller 8
Práctica: Repositorio, Servicio y Control **12**
Prueba en el Inicializador de Datos 12
Ejercicios **13**
Conceptos Clave Recordados 13
Ejercicios 13

Teoría. Relaciones Uno a Uno en JPA

Una relación Uno a Uno se produce cuando un registro de una entidad está asociado exactamente con un único registro de otra entidad (y viceversa). Por ejemplo, un Estudiante tiene asignado un único PerfilAcademico (con información detallada como biografía, dirección y expediente), y dicho perfil pertenece exclusivamente a ese estudiante.

Conceptos clave en JPA:

- **@OneToOne**: Anotación que define la propiedad de asociación entre dos entidades.
- **@JoinColumn**: Especifica la columna de clave ajena (Foreign Key) en la tabla base que establecerá el enlace físico.
- **Lado Propietario vs Lado Inverso**: La entidad que contiene la anotación @JoinColumn (y por tanto la clave ajena en su tabla SQL) es la propietaria de la relación. En el lado inverso se suele utilizar el atributo mappedBy.
- **Cascada (Cascade)**: Permite propagar operaciones automáticas (como PERSIST o REMOVE) de una entidad principal a su asociada.

Simplificando con Lombok

En el desarrollo con JPA y Spring, las entidades suelen requerir una gran cantidad de métodos auxiliares que no aportan lógica de negocio real (constructores vacíos, con parámetros, getters, setters, toString). Lombok es una librería que se integra en el momento de la compilación mediante anotaciones, generando automáticamente todo este código bajo el capó.

- **@Data**: Agrupa **@Getter, @Setter, @ToString, @EqualsAndHashCode y @RequiredArgsConstructor**.
- **@NoArgsConstructor**: Genera un constructor vacío (obligatorio para Hibernate).
- **@AllArgsConstructor**: Genera un constructor con todos los atributos de la clase.

Integración de Lombok

Añade la dependencia de Lombok en la sección de dependencias de tu archivo **pom.xml**:

```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

Dependiendo del entorno de desarrollo (IntelliJ IDEA o Eclipse), es posible que debas asegurarte de tener habilitados los "Annotation Processors" en la configuración del IDE para que Lombok funcione correctamente.

Paso a Paso: Implementación de la Relación 1a1

Vamos a modelar la relación entre **Estudiante** y una nueva entidad llamada **PerfilAcademico**. Todo ello ayudados por Lombok.

Creación de la entidad PerfilAcademico con Lombok

```
package com.educacion.demospring.model;


import jakarta.persistence.*;
	import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;


@Entity
@Table(name = "perfil_academico")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class PerfilAcademico {


   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;


   @Column(name = "biografia", length = 250)
   private String biografia;
```

```
    @Column(name = "nivel_ingles", length = 20)
    private String nivelIngles;

    // Constructor útil sin ID para nuevas instancias
        public PerfilAcademico(String biografia, String
nivelIngles) {
        this.biografia = biografia;
        this.nivelIngles = nivelIngles;
    }
}
```

Modificación de la entidad Estudiante con Lombok

Actualizamos la clase Estudiante integrando **Lombok** y la relación **@OneToOne**:

```
package com.educacion.demospring.model;
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "estudiantes")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Estudiante {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    @Column(name = "nombre", nullable = false, length = 50)
    private String nombre;

    @Column(name = "correo", nullable = false, unique = true,
length = 100)
    private String correo;

    @Column(name = "edad")
    private int edad;

    // Relación Uno a Uno con PerfilAcademico
    @OneToOne(cascade = CascadeType.ALL, fetch =
FetchType.EAGER)
    @JoinColumn(name = "perfil_id", referencedColumnName =
"id")
    private PerfilAcademico perfilAcademico;

    // Constructor auxiliar sin ID
    public Estudiante(String nombre, String correo, int edad,
PerfilAcademico perfilAcademico) {
        this.nombre = nombre;
        this.correo = correo;
        this.edad = edad;
        this.perfilAcademico = perfilAcademico;
    }
}
```

Explicación técnica

`@JoinColumn(name = "perfil_id")`: Crea físicamente una columna llamada perfil_id en la tabla estudiantes que actúa como clave foránea hacia la tabla perfiles_academicos.

`cascade = CascadeType.ALL`: Indica que al guardar o eliminar un Estudiante, la operación se propagará automáticamente a su PerfilAcademico asociado.

Actualización del Controller

También tenemos que realizar unos cambios al PUT y PATCH del Controller para añadir esta nueva relación. En EstudianteController.java:

```
// PUT: Actualización completa
@PutMapping("/{id}")
public ResponseEntity<?> actualizarCompleto(@PathVariable Long id, @RequestBody Estudiante estudianteDatos) {
   return estudianteService.buscarPorId(id)
           .<ResponseEntity<?>>map(estudianteExistente -> {

estudianteExistente.setNombre(estudianteDatos.getNombre());
               estudianteExistente.setEdad(estudianteDatos.getEdad());

               // Actualización de la relación 1 a 1
                // Si la relación ya existía y vienen datos nuevos, o si es la primera vez que se asigna
               if (estudianteDatos.getPerfilAcademico() != null) {
                   if (estudianteExistente.getPerfilAcademico() != null) {
                       // Actualizamos el expediente existente manteniendo su ID

estudianteExistente.getPerfilAcademico().setNivelIngles(

estudianteDatos.getPerfilAcademico().getNivelIngles()
                       );
```

```
estudianteExistente.getPerfilAcademico().setBiografia(

estudianteDatos.getPerfilAcademico().getBiografia()

                        );

                    } else {

                        // Asignamos un nuevo expediente

estudianteExistente.setPerfilAcademico(estudianteDatos.getPerfilAcademico());

                    }

                } else {

                    // Si se envía null en PUT, se desvincula o elimina la relación

                    estudianteExistente.setPerfilAcademico(null);

                }

                                                Estudiante actualizado = estudianteService.guardarEstudiante(estudianteExistente);

                return ResponseEntity.ok(actualizado);

            })

                                                .orElseGet(() -> ResponseEntity.status(HttpStatus.NOT_FOUND)

                        .body(Collections.singletonMap("mensaje", "El estudiante no existe")));

}


// PATCH: Actualización parcial

@PatchMapping("/{id}")

public ResponseEntity<?> actualizarParcial(@PathVariable Long id, @RequestBody Map<String, Object> campos) {

   return estudianteService.buscarPorId(id)

            .<ResponseEntity<?>>map(estudianteExistente -> {

                campos.forEach((campo, valor) -> {
```

```
                    switch (campo) {
                        case "nombre":
                                estudianteExistente.setNombre((String)
valor);
                            break;
                        case "correo":
                                estudianteExistente.setCorreo((String)
valor);
                            break;
                        case "edad":
                            if (valor instanceof Number) {
                                estudianteExistente.setEdad(((Number)
valor).intValue());
                            }
                            break;
                      case "perfilAcademico":
                            if (valor instanceof Map) {
                                @SuppressWarnings("unchecked")
                                Map<String, Object> datosExpediente =
(Map<String, Object>) valor;

                                // Si el estudiante no tenía
expediente previo, creamos uno nuevo
                                PerfilAcademico perfilAcademico =
estudianteExistente.getPerfilAcademico();
                                if (perfilAcademico == null) {
                                    perfilAcademico = new
PerfilAcademico();

estudianteExistente.setPerfilAcademico(perfilAcademico);
                                }

                                // Actualizamos solo los campos del
expediente incluidos en el JSON
```

```
                                                                    if
(datosExpediente.containsKey("biografia")) {

perfilAcademico.setBiografia((String)
datosExpediente.get("biografia"));

                                    }

                                                                    if
(datosExpediente.containsKey("nivelIngles")) {

perfilAcademico.setNivelIngles(((String)
datosExpediente.get("nivelIngles")));

                                    }

                                } else if (valor == null) {

                                    // Permitir eliminar la relación
explícitamente enviando "expediente": null

estudianteExistente.setPerfilAcademico(null);

                                }

                                break;

                        }

                    });


                                                Estudiante   actualizado   =
estudianteService.guardarEstudiante(estudianteExistente);

                    return ResponseEntity.ok(actualizado);

                })

                                                            .orElseGet(()     ->
ResponseEntity.status(HttpStatus.NOT_FOUND)

                                .body(Collections.singletonMap("mensaje", "El
estudiante no existe")));

}
```

Práctica: Repositorio, Servicio y Control

Dado que el repositorio de Estudiante ya existe y hereda de JpaRepository, este ya es capaz de gestionar automáticamente la entidad junto con su perfil gracias a la cascada.

Prueba en el Inicializador de Datos

Actualiza tu componente de arranque **InicializadorDatos.java** para verificar la relación:

Dentro del **CommandLineRunner**:

```
PerfilAcademico perfil1 = new PerfilAcademico("Apasionado del desarrollo backend", "B2");

Estudiante est1 = new Estudiante("Laura Pérez", "laura.perez@educacion.es", 21, perfil1);


estudianteRepository.save(est1);


System.out.println(">>> Estudiante con perfil guardado con éxito.");
```

Ejercicios

Conceptos Clave Recordados

- La anotación @OneToOne modela asociaciones exclusivas de uno a uno entre tablas.
- @JoinColumn define la clave foránea en la tabla propietaria de la relación.
- El uso de CascadeType.ALL evita tener que guardar las entidades dependientes de forma manual por separado.

Ejercicios

1. **Crear la Entidad Oficina**: Diseña una nueva entidad llamada **Oficina** con los atributos **id, codigoOficina** (String) y **edificio** (String).
2. **Relacionar Profesor y Oficina**: Añade una relación **@OneToOne** en la entidad **Profesor** para asociarla con una **Oficina**, configurando su respectivo **@JoinColumn** y probando su persistencia a través del flujo de la aplicación.